

Worksheet No. - 01

Student Name: Sukhleen Kaur

UID: 25MCA20031

Branch: MCA

Section/Group: 25MCA6-B

Semester: 2nd

Date of Performance: 14.01.2026

Subject Name: Design and Analysis of Algorithm Lab **Subject Code:**25CAP-612

Aim/Overview of the practical: Sort a given set of elements using the Quick sort method and determine the time required to sort the elements. Repeat the experiment for different values of n, the number of elements in the list to be sorted and plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

Objective: To implement the Quick Sort algorithm for sorting a given set of elements and to analyse its performance by measuring the time required to sort lists of varying sizes. The experiment aims to study the relationship between input size (n) and execution time by generating or reading elements from a file and plotting a graph of time taken versus n to evaluate the time complexity and efficiency of the Quick Sort method.

Procedure/Algorithm/Code

Algorithm:

1. Start
2. Read number of elements n
3. Read n elements into array
4. Apply Quick Sort:
 - Choose last element as pivot
 - Partition array so elements $\leq$ pivot are on left and $>$ pivot on right
 - Recursively apply Quick Sort to left and right subarrays
5. Display the sorted array
6. Stop

Code:

```
#include <iostream>

using namespace std;

void swap(int *a, int *b){

    int t = *a;

    *a = *b;

    *b = t;

}

int partition(int arr[], int low, int high) {

    int pivot = arr[high];

    int i = low - 1;

    for (int j = low; j <= high - 1; j++) {

        if (arr[j] <= pivot) {

            i++;

            swap(&arr[i], &arr[j]);

        }

    }

    swap(&arr[i + 1], &arr[high]);

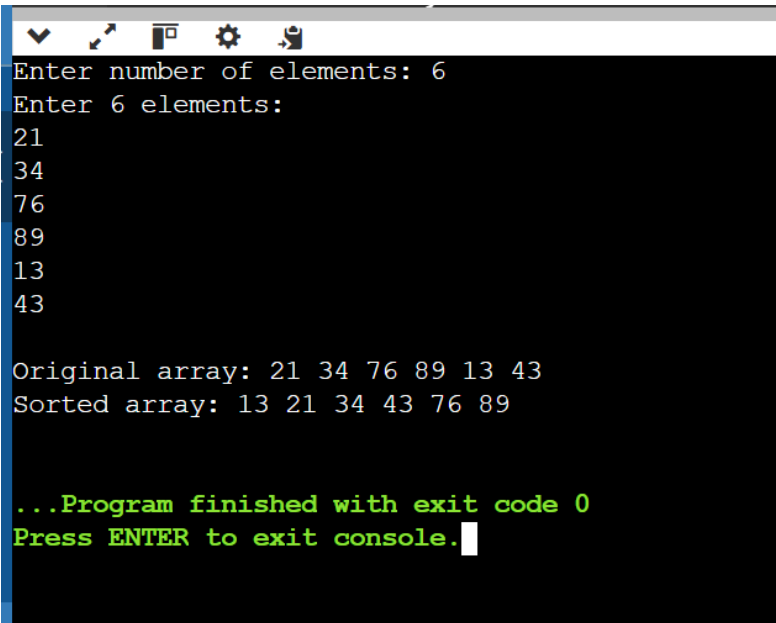
    return i + 1;

}
```

```
void quickSort(int arr[], int low, int high) {  
    if (low < high) {  
        int pi = partition(arr, low, high);  
        quickSort(arr, low, pi - 1);  
        quickSort(arr, pi + 1, high);  
    }  
}  
  
void printArray(int arr[], int size) {  
    for (int i = 0; i < size; i++)  
        cout << arr[i] << " ";  
    cout << endl;  
}  
  
int main() {  
    int n;  
    cout << "Enter number of elements: ";  
    cin >> n;  
    int arr[n];  
    cout << "Enter " << n << " elements:\n";  
    for (int i = 0; i < n; i++) {  
        cin >> arr[i];  
    }  
    cout << "\nOriginal array: ";  
    printArray(arr, n);  
    quickSort(arr, 0, n - 1);
```

```
cout << "Sorted array: ";  
  
printArray(arr, n);  
  
return 0;  
  
}
```

Output



```
Enter number of elements: 6  
Enter 6 elements:  
21  
34  
76  
89  
13  
43  
  
Original array: 21 34 76 89 13 43  
Sorted array: 13 21 34 43 76 89  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Learning outcomes (What I have learnt):

1. Understood the working of the Quick Sort algorithm.
2. Implemented Quick Sort using recursion.
3. Analyzed the time complexity of Quick Sort.